Chapter 7 : Computing scores in a complete search system

https://gemini.google.com/share/b154bcff5f23
https://chatgpt.com/share/69d2383b-e220-83e8-af95-8d1fd6a67db2
[From book]

⭐ [7.1] Efficient scoring and ranking

🔁 Quick Recap: What Are We Doing?

In Chapter 6, we ranked documents against a query using **cosine similarity** with **tf-idf weights**. The formula was:

$$\text{similarity}(q, d) = v(q) \cdot v(d)$$

where both the query and document are represented as **unit vectors** in tf-idf space.

When you search something like:

👉 **q = "jealous gossip"**

The system must:

1. Find relevant documents
2. **Score them (how relevant?)**
3. **Rank them (best first)**

The problem? Doing this naively for every document in a large collection is **slow**. Section 7.1 is about making this **faster**.

In IR, we represent:

- Query → vector **v(q)**
- Document → vector **v(d)**

Each word = a dimension

2 observations were made: the query is only 2 words + there are no weights assigned so both are equal

✅ **Key Observation 1:** Query has only 2 words → only 2 non-zero values

Example:

Term	Value
jealous	0.707
gossip	0.707
others	0

> 👉 Why 0.707?
> Because it's a **unit vector (normalized length = 1)**

[how is it 0.707 and not just 1?]

🔹 Step 1: Raw Vector (Before Normalization)

Each term appears once, so:

```
q = (1, 1)
```

🔹 Step 2: Convert to Unit Vector

In cosine similarity, we **normalize vectors** so their length = 1.

We use this formula:

$$\|q\| = \sqrt{1^2 + 1^2}$$

This gives:

```
|q| = √2 ≈ 1.414
```

🔹 Step 3: Divide Each Component

Now divide each value by the length:

$$\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

🔹 Step 4: Final Value

```
1 / √2 ≈ 0.707
```

So the normalized query vector becomes:

```
(0.707, 0.707)
```

✅ Key Observation 2: Both values are equal

Because:

- No special weighting
- Both terms equally important

⭐ Important Optimization Idea

Instead of using: 👉 **Normalized query vector** $v(q)$

We use: 👉 **Simplified vector** $V(q)$ where:

- All **non-zero values = 1**

"Query normalization can be skipped in ranking because it only scales scores by a constant and does not affect document order."

Why? Because we only care about: **Ranking (relative scores), NOT exact values**

> we prefer (1,1) over (0.707,0.707) cause its : **faster multiplication, simpler + ranking remains the same**

$$\vec{V}(q) \cdot \vec{v}(d_1) > \vec{V}(q) \cdot \vec{v}(d_2) \Leftrightarrow \vec{v}(q) \cdot \vec{v}(d_1) > \vec{v}(q) \cdot \vec{v}(d_2).$$

^ using simplified vector [V(q)] DOES NOT change the ranking!

What we're doing:

1. Looks at **query words**
2. Finds documents containing those words
3. **Adds up scores** for each document
4. Returns the **top K best documents**

Cosine similarity becomes: 👉 **Sum of term weights in the document**

Since query weights = 1

- So instead of: score += wt,q × wt,d
- We do: **score += wt,d**

👉 **Faster (no multiplication!)**

> example:

📂 Documents (from postings list)

Document	wt(jealous,d)	wt(gossip,d)
D1	3	2
D2	1	4

🧮 Method 1: ORIGINAL Formula

```
score += wt,q × wt,d
```

Since:

```
wt,q = 1
```

🔹 **D1:**

```
= (1 × 3) + (1 × 2)
= 3 + 2 = 5
```

🔹 **D2:**

```
= (1 × 1) + (1 × 4)
= 1 + 4 = 5
```

⚡ Method 2: SIMPLIFIED Formula

```
score += wt,d
```

🔹 **D1:**

```
= 3 + 2 = 5
```

🔹 **D2:**

```
= 1 + 4 = 5
```

💥 SAME RESULT!

👉 Multiplication disappeared
👉 Answer stayed the same

› using inverted index:

- Instead of scanning all doc; we only look at docs containing the query terms
- The system goes through the postings lists of each query term one by one. As it encounters a document in these lists, it adds the term's weight (like tf or tf-idf) to that **document's total score.**
- We only **compute scores** for **documents** that **appear in at least one** of the postings lists of the query terms**.**
- If the total number of documents is **N**, and only **J** documents contain the query terms, then we only score those **J documents**, where typically **J is much smaller than N**. This is what makes the method efficient.

Process:

For each term in query:

1. Go to its **postings list**
2. For each document:
    - Add its score

- the system uses two important values:
    - **tf (term frequency)** → how often a word appears in a **document**

$$tf_{t,d} = \text{number of times term } t \text{ appears in document } d$$

$$tf_{t,d} = \frac{\text{number of times term } t \text{ appears in } d}{\text{total number of terms in document } d}$$

⚠️ Sometimes (important!)

We don't use raw tf. We use log-scaled tf:

$$tf_{t,d} = 1 + \log(\text{frequency})$$

👉 Why?

To reduce the effect of very frequent words.

- **idf (inverse document frequency)** → how rare/important the word is across all documents

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

💡 Meaning

- **N** = total number of documents
- **df_t** = number of documents containing term t

example walkthrough:

📊 Example

Let's say:

- Total documents N = 1000

Term	df	idf
jealous	10	high
gossip	100	medium
the	900	very low

👉 "jealous" is rare → more important 🔥

🧠 3. TF-IDF (Combined Weight)

📌 Formula:

$$w_{t,d} = tf_{t,d} \times idf_t$$

💡 Meaning

This is the **actual score we add** in the algorithm.

👉 Combines:

- importance in document (tf)
- importance in collection (idf)

📊 Full Example

Let's say:

- N = 1000
- df(jealous) = 10 → idf ≈ log(1000/10) = log(100)
- df(gossip) = 100 → idf ≈ log(10)

Document D1:

```
"jealous gossip gossip"
```

- tf(jealous) = 1
- tf(gossip) = 2

Compute weights:

- w(jealous, D1) = 1 × log(100)
- w(gossip, D1) = 2 × log(10)

Final score (for query):

```
Score(D1) = w(jealous) + w(gossip)
```

› Finding Top K Documents

- After all scores are computed, the system needs to return only the **top K documents**
- Sorting all docs by score is TOO SLOW
- we use a **heap (priority queue)**, which is a data structure designed to efficiently retrieve the largest values.
- **Building** the heap takes about **2J comparisons**, and then **extracting** each of the top K documents takes **log J time**. [J → no of docs containing query terms]

> Algorithm:

FastCosineScore(q)
1 float $Scores[N] = 0$
2 **for each** d
3 **do** Initialize $Length[d]$ to the length of doc d
4 **for each** query term t
5 **do** calculate $w_{t,q}$ and fetch postings list for t
6 **for each** pair(d, $tf_{t,d}$) in postings list
7 **do** add $wf_{t,d}$ to $Scores[d]$
8 Read the array $Length[d]$
9 **for each** d
10 **do** Divide $Scores[d]$ by $Length[d]$
11 **return** Top K components of $Scores[]$

Figure 7.1 A faster algorithm for vector space scores.

[1] Initialize scores

[2] Loop through all documents in the collection

[3] Initialize Length[d]:
For each document, store its length (used later for normalization).

[4] Go through each word in the query one by one.

[5] Calculate wt,q and fetch postings list:
Compute the query term weight (usually treated as 1 here) and retrieve the postings list of term t (list of documents containing t).

[7] Add term weight to Scores[d]:
For each (document d, term frequency tft,d) pair in the postings list, add the document's term weight to its score → `Scores[d] += wt,d`

[8] Read Length[d]: Access the stored document lengths for normalization.

[9] For each document d:
Loop through all documents that have accumulated scores.

[10] Normalize scores:
Divide each document's score by its length →

```
Scores[d] = Scores[d] / Length[d]
```

> To ensure long documents don't win just by having more words, the **total score is divided by the document's length (Cosine Normalization).**

[11] Select and return the K documents with the highest scores.

⭐[7.1.1] Inexact Top K Document Retrieval

- **Exact Top K** → find *the* **absolute best K** *documents*
- **Inexact Top K** → find *documents that are* **very close to the best K**

- **trading precision for speed**!!

- Earlier, we assumed that the system computes scores for documents and returns exactly the top K highest-scoring ones. However, in reality, this is expensive because it may require computing scores for a large number of documents
- In **inexact retrieval**, instead of guaranteeing the perfect top K, the system returns documents that are **very likely to be among the best**.

> Why Inexact Retrieval is Acceptable

- Cosine similarity is only an approximation of relevance
- Users **cannot distinguish tiny differences in scores**
- Speed is more important than perfect accuracy

The ranking function (cosine similarity) is not a perfect measure of what the user actually wants — it is just a mathematical approximation. Even if we return the mathematically "top" documents, they may not truly be the most relevant from a human perspective. Therefore**, returning documents that are almost as good is usually sufficient.**

› Main Goal of This Section

- Reduce computation cost
- Avoid scoring too many documents
- Still return high-quality results

This section introduces strategies (**heuristics**) that help us **ignore many documents early**, focusing only on promising ones.

› The Core Idea: Reduce N to a Smaller Set

- N = total documents
- J = docs containing query terms
- **A** = **smaller subset of J** (**contenders**)

A ⇒ This set contains documents that are *likely* to be among the best. The idea is to drastically reduce the number of documents we process, which improves speed while still maintaining good quality results.

› Two-Step Scheme

✅ Step 1: Find Contender Set A

- A is a subset of documents
- K < |A| << N
- Contains likely good candidates

A is **not guaranteed to include the exact top K**, but it is designed to include many high-scoring documents. The selection is based on heuristics (smart rules).

✅ Step 2: Select Top K from A

- **Compute scores only for A**
- **Use heap or sorting**
- **Return best K**

Since A is much smaller than the full dataset, this step is much faster than evaluating all documents. Not only do we compute fewer scores, but we also **reduce the size of the heap used for ranking**, making the entire process faster.

› heuristics

- Rules or strategies to filter documents
- Not exact, but effective
- Need tuning

example Heuristic: “Only consider documents where query terms appear frequently”

› Not Suitable for All Query Types

- inexact retrieval methods are designed for **free-text queries**, where **ranking** is **flexible**.
- However, for **Boolean queries** (like AND/OR) or **phrase queries** ("exact match"), **precision is critical**, so we cannot skip documents or approximate results.

⭐ [7.1.2] Index Elimination

- Index elimination is about **reducing the search space early** so we don't waste time scoring too many documents.
- Even after using an inverted index, there can still be thousands of documents to process. So instead of considering all of them, we apply additional rules (heuristics) to **eliminate less useful documents or terms before scoring**. This makes the system significantly faster while still maintaining good result quality

🔹 Heuristic 1: Ignore Low-IDF Terms

- Only process terms with **high idf** / terms whose idf **exceeds a certain threshold.**
- Ignore common words (low idf)
- Treat them like stop words

- first idea is to **ignore query terms** that are **too common** across documents. These terms have **low idf**, meaning they appear in many documents and therefore are not very useful for distinguishing between relevant and irrelevant results. By skipping these terms entirely during postings traversal, we avoid processing very large postings lists
- This significantly reduces the number of documents we need to score, improving efficiency without losing much relevance.

Low-idf terms ≈ stop words

- **Adjustable threshold** → IDF threshold can change per query
- The cutoff for what counts as "low idf" is not fixed
- if a query has very few terms, we may lower the threshold to avoid losing too much information

Query:

```
"catcher in the rye"
```

IDF intuition:

Term	Type
catcher	rare → high idf ✅
rye	rare → high idf ✅
in	common → low idf ❌
the	very common → low idf ❌

- “the”, “in” → appear in almost every doc

- Their postings lists are HUGE

- Skipping them **saves massive computation**

🔹 Heuristic 2: Require Many Query Terms in Document

- Only consider documents containing:
  - **many query terms**
  - **or all query terms**

- The intuition is that **documents containing more query terms are more likely to be relevant.**
- So during postings traversal, we can **filter out documents that match only one term and only keep those that match several (or all) query terms.**
- This further reduces the number of documents we need to score.

⁉️ PROBLEM:

- Might eliminate too many documents
- May end up with fewer than K results

If we require documents to contain all or many query terms, we might filter out too many documents. As a result, we may not even have enough documents left to return the top K results. This is a serious issue because users expect at least K results.

⭐[7.1.3] Champion lists / Fancy lists / top docs

- For each term, store only its **top r documents**
- These documents have the **highest weights (tf or tf-idf)**
- **Precomputed** during indexing

Some documents contain that term more frequently or more prominently, making them more relevant. So instead of storing or processing all documents for a term during query time, we **precompute a smaller list of the best documents** for each term. These are called **champion lists**

› What exactly is a Champion List?

- For each term **t**
- Store top **r documents**
- Based on highest **tf or tf-idf weight**

This list is stored **in advance**, so during query processing we don't have to look at all documents — only the best ones.

Term: "gossip"

All documents:

Doc	tf
D1	10
D2	2
D3	8
D4	1
D5	6

If r = 3, champion list =

```
D1, D3, D5
```

👉 Only top 3 documents kept

› How It Works During Query:

For query q

1) Take **union of champion lists** of each of its terms
2) Call this set **A**
3) Only score documents in A

Why this works? High-weight docs are most relevant and Low-weight docs unlikely to be top results, so it's safe to ignore them

› Important Parameter: r

- r = size of champion list
- Must be chosen carefully
- Usually **r > K**

› **Problem:** r is Fixed but K is Dynamic

- r chosen at **indexing** time [**fixed**]
- K chosen at **query** time [**variable**]
- May not match

- Since we only consider documents in champion lists, it is possible that the total number of candidate documents (set A) is smaller than K. In such cases, we cannot return enough results, which is a problem.
- Systems must handle this by either **increasing r** or **falling back to full postings.**

› Different r for Different Terms

- r can vary per term
- **Rare terms → larger r**
- **Common terms → smaller r**

› **Rare terms (high idf)** are **more important**, so we may **want to include more documents for them (larger r).**
› **Common terms (low idf)** are **less useful**, so a smaller r may be sufficient.

⭐[7.1.4] Static Quality Scores and Ordering

› What is a Static Quality Score g(d)?

- A **query-independent score** for each document
- Value between **0 and 1**
- Measures overall quality of a document

> Some documents are **inherently better** than others, regardless of the query.

> For example, a well-written, highly trusted article is generally better than a low-quality one. This is captured using a **static quality score g(d).**

> It is called **static** because it **does not depend on the query** — it is **precomputed once** and **reused** for all searches.

Document	Quality g(d)
Doc1	0.25 (low quality)
Doc2	0.5 (medium)
Doc3	1 (high quality)

> Combining Quality with Relevance

Final score = quality + relevance (cosine similarity)

- Relevance = **cosine similarity**
- Both contribute to ranking

document should not only match the query, but also be of good quality.

So we combine:

- **Query-dependent score** → how relevant it is
- **Static quality score** → how good the document is overall

$$\text{net-score}(q, d) = g(d) + \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)||\vec{V}(d)|}.$$

📊 Example

Doc	Cosine Score	g(d)	Final Score
D1	0.9	0.2	1.1
D2	0.7	0.8	1.5

👉 Even though D1 is more relevant,
👉 D2 wins because it's higher quality

› Ordering Postings by Quality:

- **Sort postings** by **g(d)** instead of docID
- Same ordering across all lists
- Enables efficient traversal

Now we sort postings by **static quality score g(d)** (**highest first**). This means that when we traverse postings, we encounter **high-quality documents first**, increasing the **chance of finding top results early.**

📊 Understanding the Figure (7.2)

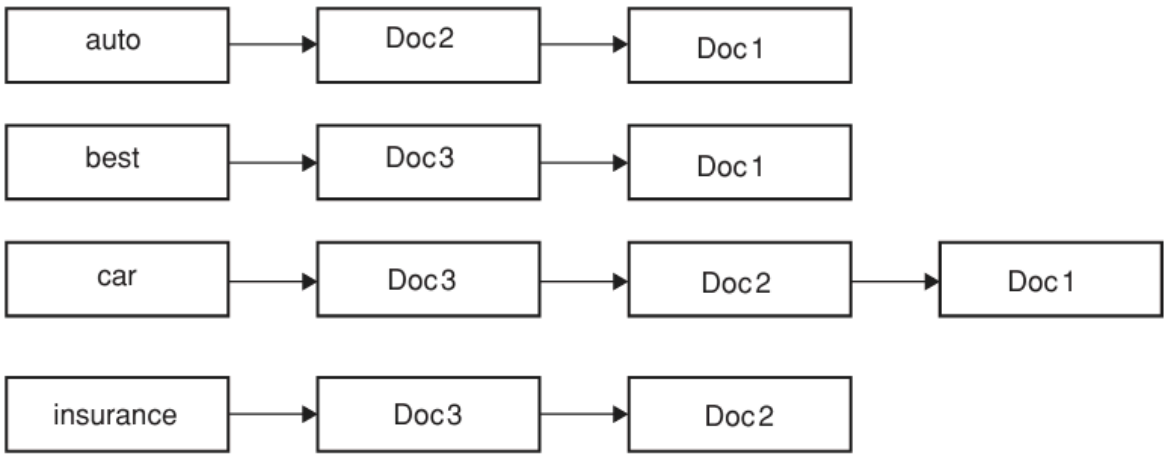

Figure 7.2 A static quality-ordered index. In this example we assume that Doc1, Doc2 and Doc3 respectively have static quality scores $g(1) = 0.25$, $g(2) = 0.5$, $g(3) = 1$.

Given:

- g(Doc1) = 0.25
- g(Doc2) = 0.5
- g(Doc3) = 1

👉 Sorted by **g(d)** descending: **Doc3 (1) > Doc2 (0.5) > Doc1 (0.25)**

> Global Champion Lists (Extension)

Combine:

- quality (g(d))
- tf-idf

Keep top r documents per term

> improve champions list by also considering document quality alongside tf-idf. For each term, we store the top r documents based on:

g(d) + tf-idf

This ensures we keep documents that are:

- Relevant to the term
- AND high quality

 # Query Processing with Global Champion Lists

- Take union of champion lists
- Compute scores only for them
- Faster retrieval

At query time, we only consider documents from these improved champion lists. Since these lists already contain high-quality and relevant documents, we can safely ignore the rest, reducing computation while maintaining accuracy.

> Two-Level Lists: High and Low

For each term:

- **High list** → top m important docs
- **Low list** → remaining docs

Instead of treating all documents equally, we split postings into two parts:

1. **High list** → best documents (high tf or importance)
2. **Low list** → less important documents

At query time:

- First, process only the **high lists**
- If we get enough results (K), stop early
- Otherwise, continue with low lists

⭐ [7.1.5] Impact Ordering

What Was Happening Before?

- All postings lists had a **common order** (docID or quality)
- Allowed **document-at-a-time scoring**

Because of this consistent ordering, we could traverse all posting lists **together (in parallel)** and compute scores document by document. This approach is called **document-at-a-time scoring**, where we process one document at a time and compute its full score using all query terms.

> What Changes in Impact Ordering?

- Postings are sorted by **term freq (tf)**
- No common ordering across lists
- Switch to **term-at-a-time scoring**

- In impact ordering, instead of sorting postings by docID or quality, we **sort** each postings list by **decreasing term frequency (tf$_{t,d}$)**.
- This means documents where the term appears most frequently come first.
- However, this breaks the common ordering across postings lists — each term now has its own different order. Because of this, we **can no longer traverse all lists together.**
- Instead, we process **one term at a time**, updating scores for documents as we go. This is called **term-at-a-time scoring.**

› Idea 1: Process Only a Prefix of Postings

- Don't process full postings list
- Stop early:
    - after **r documents**, OR
    - when tf drops below threshold

› Idea 2: Process Query Terms by IDF

- Process **high-idf terms first**
- These contribute more to score

Rare terms (high idf) carry more weight and have a bigger impact on scoring. So we process those terms first. This allows us to quickly identify strong candidate documents. If later terms (low idf) contribute very little, we may skip them entirely.

Query:

```
"the gossip scandal"
```

Term	IDF
scandal	high
gossip	medium
the	very low

👉 Processing order:

```
scandal → gossip → the
```

› Adaptive Stopping

- Check if scores improve significantly
- If not → stop early

- As we process query terms one by one, we observe how much the document scores change.
- If **adding another term does not significantly change scores**, it means that term is **not very useful**.
- In that case, we can stop processing further terms or process only a small portion of their postings. This makes the system **adaptive** and more **efficient**.

› General Idea: Focus on High Impact

Prioritize:

- **high tf**
- **high idf**

Ignore low-impact contributions

› Generalization

- Although the basic idea uses term frequency (tf), we can generalize this approach by ordering postings based on any measure of importance — such as **tf-idf** or a **combination of static quality and term relevance**
- When postings are ordered based on their **contribution to the score**, this general approach is called **impact ordering**

› Key Tradeoff

- Faster retrieval ✅
- Not exact results ❌

⭐ [7.1.6] Cluster Pruning

- **Group documents into clusters**
- At query time → search only **one (or few) clusters**
- Avoid scanning all documents

based on the idea that **similar documents are close to each other** in **vector space**. So instead of searching through all documents, we first group them into clusters.

Then, when a query arrives, we **find the cluster that is closest to the query and only search within that cluster**. This drastically reduces the number of documents we need to score, making retrieval much faster.

› Preprocessing Step (Clustering)

📌 Step 1: Choose Leaders

- Pick **√N documents randomly**
- These are called **leaders**

These leaders act as **representatives of clusters**.

these leaders will be spread across the document space, roughly covering different regions.

📌 Step 2: Assign Followers

- Remaining documents = **followers**
- Each follower is assigned to its **nearest leader**

All remaining documents (called followers) are **assigned to the closest leader** based on **similarity** (usually cosine similarity).

This creates clusters where each leader has a group of followers that are similar to it.

Size intuition:

- Number of leaders = √N
- Followers per leader ≈ √N

If we have N documents and √N leaders, then on average each leader will have about √N followers. This creates **balanced clusters**, which is ideal for efficient searching.

› Query Processing Step

- Compare query with all leaders
- Choose closest leader L

A set is formed

- **A = leader + its followers**
- Only these documents are considered

📊 Example

- Query: "car insurance"
- Closest leader: L2

Cluster:

```
L2 → D5, D8, D12, D20
```

👉 Candidate set:

```
A = {L2, D5, D8, D12, D20}
```

👉 Only these are scored

Why Random Leaders Work;

› **Dense** regions (many similar documents) are likely to contain **more leaders**, resulting in finer clusters.
› **Sparse** regions will have **fewer** leaders.

This makes the clustering surprisingly effective despite its simplicity.

› Improved Version with Parameters (b_1, b_2)

- **b_1** → number of leaders per follower
- **b_2** → number of leaders checked at query time

To improve accuracy, we can **assign each follower to multiple leaders** (**b_1 closest leaders instead of just one).**
Similarly, at query time, instead of choosing just one leader, we **can consider the top b_2 closest leaders**.
This increases the number of candidate documents, improving the chances of finding the true top K results.

📊 Example

- $b_1 = 2$ → each document belongs to 2 clusters
- $b_2 = 3$ → query checks 3 leaders

👉 Candidate set becomes larger and more accurate

- **Tradeoff?**
    - More clusters checked → better accuracy
    - But more computation

⭐📊 Comparison of Efficient Retrieval Techniques

Technique	Core Idea	How It Reduces Work	Key Data Structure / Trick	Advantages	Disadvantages
Index Elimination (7.1.2)	Ignore unimportant terms/docs	Skip low-idf terms and docs with few query terms	IDF threshold, term filtering	Very simple, reduces large postings lists	May remove useful documents; risk of < K results
Champion Lists (7.1.3)	Precompute top r docs per term	Only score docs in champion lists	Top r docs by tf or tf-idf	Fast at query time, focuses on strong docs	r fixed → may miss good docs; may return < K
Static Quality + Ordering (7.1.4)	Use document quality score g(d)	Process high-quality docs first	Sort postings by g(d), combine with tf-idf	Better ranking (quality + relevance), early good results	Requires extra storage/computation; still may scan many docs
Impact Ordering (7.1.5)	Process high-impact contributions first	Only process top part of postings lists	Sort postings by tf (or impact)	Very efficient; early stopping possible	No common order → more complex; inexact results
Cluster Pruning (7.1.6)	Group docs into clusters	Only search one (or few) clusters	Leaders (√N), followers	Huge reduction (N → √N), very fast	May miss relevant docs in other clusters

🔹 1. What They Eliminate

Technique	Eliminates
Index Elimination	Terms + weak documents
Champion Lists	Low-weight documents per term
Static Quality	Low-quality documents (implicitly)
Impact Ordering	Low-impact parts of postings
Cluster Pruning	Entire groups of documents

🔹 2. When Filtering Happens

Technique	Stage
Index Elimination	Query time
Champion Lists	Preprocessing + query time
Static Quality	Preprocessing + query time
Impact Ordering	Query time
Cluster Pruning	Preprocessing + query time

🔹 3. Type of Strategy

Technique	Strategy Type
Index Elimination	Filtering terms/docs
Champion Lists	Precomputed top candidates
Static Quality	Better ranking + ordering
Impact Ordering	Smart traversal + early stopping
Cluster Pruning	Divide-and-search (clustering)

⭐ [book qs]

✅ Exercise 7.1

Why decreasing order of g(d)?

👉 We use **decreasing order** so that **high-quality documents are processed first**, increasing the chance of finding top K results early and enabling early termination.

✅ Exercise 7.2

Why tf for champion lists but tf-idf + g(d) for global champion lists?

👉 Basic champion lists focus only on **term importance within a document (tf).**
👉 Global champion lists consider **overall importance (tf-idf) + document quality g(d),** giving a more accurate ranking across queries.

✅ Exercise 7.3

Why r = K works for one-term queries? What about s-term queries?

👉 For **one-term queries**, ranking depends only on that term → top K docs for that term = correct answer.

👉 For **s-term queries**, use:

- **r > K** (larger candidate set), OR
- Take **union of champion lists of all terms**

✅ Exercise 7.4

How does common ordering by g(d) help efficiency?

👉 Since all postings are sorted by the **same order (g(d))**, we can:

- Traverse lists **simultaneously (document-at-a-time)**
- Avoid random jumps → faster scoring

✅ Exercise 7.5

(Conceptual answer only, since full numeric data not provided)

👉 Net score:

$$\text{score}(d) = g(d) + \text{cosine}(q, d)$$

👉 Doc3's rank depends on its **g(d)** relative to Doc1 and Doc2:

- **1st** → if g(D3) is large enough to exceed both
- **2nd** → if between Doc1 and Doc2
- **3rd** → if smallest

👉 So ranking changes as **g(D3) increases/decreases relative to others**

✅ Exercise 7.6

Frequency-ordered postings (concept)

👉 Instead of docID order:

- Sort postings by **decreasing tf**

Example:

term → D1(10), D3(5), D2(2)

👉 Highest frequency first

✅ Exercise 7.7

Impact ordering with g(d) + normalized tf

Euclidean normalization comes from cosine similarity:

$$w_{t,d} = \frac{tf_{t,d}}{\sqrt{\sum (tf_{t,d})^2}}$$

👉 For each posting:

$$\text{impact} = g(d) + tf_{normalized}$$

👉 Sort postings in **descending order of this value**

Example: term → D3 (highest impact) → D2 → D1

✅ Exercise 7.8 [samajh nhi aya]

Cluster pruning failure example

👉 Setup:

- Two leaders: L1 and L2
- Query Q is closer to L1
- But actual nearest point belongs to cluster of L2

📊 Example (2D plane)

- L1 cluster: points near (0,0)
- L2 cluster: points near (10,10)

Query:

Q = (5,5)

👉 Closest leader → L1
👉 But actual closest point → from L2 cluster

👉 Cluster pruning checks only L1 cluster → **wrong answer**